

LAB 4 ADVANCED MARKOV MODELS

Exercise 1: Try the code. What is the most likely sequence of hidden states? Try a different sequence of observations and infer the most likely sequence of hidden states.

with observations = np.array([[0, 1, 2, 3, 4, 5]]).T

the output terrains = [0 1 0 1 2 2]

with observations = np.array([[0, 2, 2, 1, 5, 4,5,3]]).T

the output terrains are = [0 0 0 1 2 2 2 1]

in conclusion observations 0 and 2 infers at terrain 0 = smooth surface

observations 1 and 3 infers terrain 1 = rough surface

observations 4 and 5 infers terrain 2 = slippery surface.

Exercise 2: Try the above code and write the ranked list of likely states after each observation.

```
jin@kitty:~/decision_making$ /bin/pyth
[[0.96599462 0.03400538 0.
 [0.36775461 0.63224539 0.
 [0.77842062 0.06437652 0.15720286]
 [0.15399853 0.59837062 0.24763084]
 [0.06975433 0.06062063 0.86962504]
 [0.06201751 0.19204555 0.74593694]]
```

After 1st observation from higher to lower probability, we have terrain 0, terrain 1, and 2.

After 2nd observation from higher to lower probability, we have terrain 1, terrain 0, and 2.

After 3rd observation from higher to lower probability, we have terrain 0, terrain 2, and 1.

After 4th observation from higher to lower probability, we have terrain 1, terrain 2, and 0.

After 15th observation from higher to lower probability, we have terrain 2, terrain 0, and 1.

After 6th observation from higher to lower probability, we have terrain 2, terrain 1, and 0.

Exercise 3: Try the above code. Now, remove the last 3 observations and compute the log probability for the observations [0,1,2]. Now, use just the first observation [0] and compute the log probability again. How are the log probabilities changing? Why?

All observations output = -10.852225158676852.

observations [0,1,2] output = -5.083631884064014.

observation[0] output = -1.8078888511579385.

With more observations there's more uncertainty. If you always jump from one state to another, which shouldn't be observed, if you reduce the number of observations the probability also reduces, but for example if i maintain the number of observations high in the following sequence.

observations = np.array([[2,2,2,2,2,2,2]]) the output = -5.817400071588748.

This also depends on remission probabilities.

Exercise 4: Define the HMM model. Initialize the transition and emission probabilities for the model, reflecting reasonable assumptions about how the robot's states change and the likelihood of different sensor readings.

```
model.transmat_ = np.array([
    [0.3, 0.5, 0.2], # alone -> [alone, acoompanied, interacting]
    [0.3, 0.2, 0.5], # acoompanied -> [alone, acoompanied, interacting]
    [0.3, 0.5, 0.2]  # interacting -> [alone, acoompanied, interacting]
])
```

The logic is that when the robot is with company it is because they wanna interact, and after they interact they leave.

```
# Possible observations are combinations of body face and voice detected
# O_body = {yes,no}
# O_face= {yes,no}
# O_voice = {yes,no}
# Encoding multiple discrete observations (O_body x O_face x O_voice):
# O_0 = {body_yes,face_yes,voice_yes}--> state 3 (interacting)
# O_1 = {body_yes,face_yes,voice_no}--> state 2 (accompanied)
# O_2 = {body_yes,face_no,voice_yes}--> state 3 (interacting)
# O_3 = {body_yes,face_no,voice_no}--> state 2 (accompanied)
# O_4 = {body_no,face_yes,voice_yes}--> state 1 (alone) it's impossible
# O_5 = {body_no,face_yes,voice_no}--> state 1 (alone) it's impossible
# O_6 = {body_no,face_no,voice_yes}--> state 1 (alone) it's impossible
# O_7 = {body_no,face_no,voice_no}--> state 1 (alone)

# Emission probabilities
model.emissionprob_ = np.array([
    [0.02,0.08,0.02, 0.08, 0.2, 0.2, 0.2, 0.2], # alone -> [O_0, O_1, O_2, O_3]
    [0.15,0.3,0.15, 0.3, 0.025, 0.025, 0.025, 0.025], # acoompanied -> [O_4, O_5, O_6, O_7]
    [0.4,0.05,0.4, 0.05, 0.025, 0.025, 0.025, 0.025] # interacting
])
```

So this is just if the robot doesnt sense a body, then there is no one to interact with and he is alone even if the sensor reads a face or a voice, also if the robot is in company, it needs also

to receive voice commands to start the interaction so even if someone is looking at the robot face to face, it will not start an interaction until someone talks, it doesn't need to be the same person standing in front (case 0_2).

Exercise 5: Simulate a sequence of observations and use the HMM to predict the most probable hidden states based on these observations. What is the most likely future hidden state and observations? Explain how you have obtained your answer.

with : `observations = np.array([[4,7,1,3,2,1,6,7]]).T`

the output was :

```
post_prob:
[[0.96927309 0.01558202 0.01514489]
 [0.81108168 0.11979204 0.06912628]
 [0.22404451 0.6602238 0.11573169]
 [0.1322768 0.74742816 0.12029503]
 [0.03165961 0.1453959 0.82294449]
 [0.14348388 0.77672256 0.07979356]
 [0.77419355 0.08612362 0.13968283]
 [0.77419355 0.15295578 0.07285067]]
```

The model works out the hidden states pretty accurately, after these observations using `post_prob = model.predict_proba(observations)`, the model believes it will stay alone.

Exercise 6: Simulate 10 iterations of Maggie's decision-making system. For each iteration, emulate the sensorial information by generating new observations based on the Emission Probabilities matrix and predict the most probable hidden state. Then, Maggie has to execute the corresponding action by printing a message.

Python file EX6.py

Exercise 7: Complete the gaps in the code (search for "To Be Completed HERE") and execute it.

Python file EX7.py

Exercise 8: Increment the number of steps in `test_planner` and explain the evolution of the belief state values.

nsteps = 3

```
** Testing value iteration **
==== Step 1 ====
True state: shelf1
Belief: {ObjectState(shelf1): 0.25, ObjectState(shelf2): 0.25, ObjectState(shelf3): 0.25, ObjectState(shelf4): 0.25}
Action: inspect
Reward: -10.0
>> Observation: not_found
==== Step 2 ====
True state: shelf1
Belief: {ObjectState(shelf1): 0.31481481481477, ObjectState(shelf2): 0.05555555555555555, ObjectState(shelf3): 0.31481481481477, ObjectState(shelf4): 0.31481481481477}
Action: move_right
Reward: -1.0
>> Observation: not_found
==== Step 3 ====
True state: shelf2
Belief: {ObjectState(shelf1): 0.04491413474240424, ObjectState(shelf2): 0.41215323645970947, ObjectState(shelf3): 0.11624834874504628, ObjectState(shelf4): 0.42668428005284015}
Action: inspect
Reward: 50.0
>> Observation: found
```

nsteps = 5

```
==== Step 2 ====
True state: shelf1
Belief: {ObjectState(shelf1): 0.31481481481477, ObjectState(shelf2): 0.05555555555555555, ObjectState(shelf3): 0.31481481481477, ObjectState(shelf4): 0.31481481481477}
Action: move_right
Reward: -1.0
>> Observation: not_found
==== Step 3 ====
True state: shelf2
Belief: {ObjectState(shelf1): 0.04491413474240424, ObjectState(shelf2): 0.41215323645970947, ObjectState(shelf3): 0.11624834874504628, ObjectState(shelf4): 0.42668428005284015}
Action: inspect
Reward: 50.0
>> Observation: found
==== Step 4 ====
True state: shelf2
Belief: {ObjectState(shelf1): 0.025601156550318256, ObjectState(shelf2): 0.7840672712900945, ObjectState(shelf3): 0.04750488829720848, ObjectState(shelf4): 0.1428266838623788}
Action: inspect
Reward: 50.0
>> Observation: found
==== Step 5 ====
True state: shelf2
Belief: {ObjectState(shelf1): 0.012832264932781392, ObjectState(shelf2): 0.9336345132614308, ObjectState(shelf3): 0.01721975579217544, ObjectState(shelf4): 0.03631346601361247}
Action: inspect
Reward: 50.0
>> Observation: found
```

nsteps = 7

```

==== Step 1 ====
True state: shelf1
Belief: {ObjectState(shelf1): 0.25, ObjectState(shelf2): 0.25, ObjectState(shelf3): 0.25, ObjectState(shelf4): 0.25}
Action: inspect
Reward: -10.0
>> Observation: not_found
==== Step 2 ====
True state: shelf1
Belief: {ObjectState(shelf1): 0.31481481481477, ObjectState(shelf2): 0.05555555555555555, ObjectState(shelf3): 0.31481481481477, ObjectState(shelf4): 0.31481481481477}
Action: move_right
Reward: -1.0
>> Observation: not_found
==== Step 3 ====
True state: shelf2
Belief: {ObjectState(shelf1): 0.04491413474240424, ObjectState(shelf2): 0.41215323645970947, ObjectState(shelf3): 0.11624834874504628, ObjectState(shelf4): 0.42668428005284015}
Action: inspect
Reward: 50.0
>> Observation: found
==== Step 4 ====
True state: shelf2
Belief: {ObjectState(shelf1): 0.025601156550318256, ObjectState(shelf2): 0.7840672712900945, ObjectState(shelf3): 0.04750488829720848, ObjectState(shelf4): 0.1428266838623788}
Action: inspect
Reward: 50.0

```

It seems for me the limit with horizon set to 3 is 7 steps, with more steps i need to reduce the horizon level.

If I'm being honest I don't understand the belief values, because when it inspects a place and the object is not there the belief punctuation doesn't drop too low compare to others, we see in step 2 that belief for shelf1 is still 0.31 compared to shelf 2 which is 0.05.

We also see that the first steps are always the same no matter the steps.